Лабораторна робота 6. Основи Blazor

Початок роботи з Blazor

У цій роботі ви дослідите Razor-компоненти, зміните лічильник і створите застосунок зі списком завдань. Ви навчитеся передавати параметри компонентам, обробляти натискання кнопок і прив'язувати поля введення до даних C#.

Як позначено зміни коду

Зелене виділення з вихідного документа відтворено блоками `diff`:

- рядок із `+` потрібно додати;
- рядок із `–` потрібно видалити або замінити;
- рядок без цих позначок наведено для контексту.

Знаки `+` і `–` на початку рядків `diff` не є частиною коду. Після позначених змін наведено звичайний лістинг, придатний для копіювання. Так зміни залишаються зрозумілими навіть у переглядачі без кольорового підсвічування.

Про версії шаблону та зображення

Усі зображення з DOCX збережено без змін. Вони показують старіший шаблон **Blazor Server App**, тому вигляд вікон, меню та назви файлів можуть відрізнятися від сучасного шаблону **Blazor Web App**.

Основні кроки нижче узгоджено із Blazor Web App на .NET 10 із параметрами **Server** і **Per page/component**. Старі назви наведено для зіставлення:

Призначення	Старий Blazor Server App	Blazor Web App
Головна сторінка	`Pages/Index.razor`	`Components/Pages/Home.razor`
Лічильник	`Pages/Counter.razor`	`Components/Pages/Counter.razor`
Список завдань	`Pages/Todo.razor`	`Components/Pages/Todo.razor`
Меню	`Shared/NavMenu.razor`	`Components/Layout/NavMenu.razor`
Спільні Razor-директиви	`_Imports.razor`	`Components/_Imports.razor`

У сучасному шаблоні для обробки подій на окремій сторінці потрібна серверна інтерактивність — у відповідних прикладах додано `@rendermode InteractiveServer`. У старому Blazor Server App цієї директиви не було. Документація щодо режимів рендерингу.

Що потрібно знати перед виконанням роботи

- Базові поняття вебзастосунків і HTML.
- Основи C# та .NET.
- Основи роботи з командним рядком.
- На комп'ютері має бути встановлений .NET SDK.
- Потрібен редактор: сумісна з вибраним SDK версія Visual Studio з компонентами веброзробки або Visual Studio Code з підтримкою C#.

Не потрібно одночасно використовувати обидва редактори: виконуйте всі кроки в одному з них.

Коротко про Blazor

Blazor — фреймворк платформи .NET для побудови користувацького інтерфейсу з компонентів. Залежно від моделі виконання C#-логіка компонентів може працювати на сервері або на клієнті. HTML і CSS визначають структуру та оформлення інтерфейсу.

Компонентний підхід Blazor подібний до підходів Angular, React і Vue: інтерфейс складається з окремих частин, які можна повторно використовувати. Для багатьох інтерактивних дій достатньо C#, але JavaScript залишається частиною вебплатформи та за потреби доступний через JS interop.

Blazor надає такі можливості:

- написання логіки інтерфейсу мовою C#;
- використання сумісних бібліотек екосистеми .NET;
- спільне використання частини логіки клієнтським і серверним кодом;
- використання шаблонів проєктів та інструментів розробки Visual Studio або .NET CLI.

Основні моделі виконання:

- **Blazor Server / Interactive Server:** C#-логіка інтерфейсу виконується на сервері ASP.NET Core, а браузер обмінюється з ним подіями й оновленнями через SignalR.
- **Blazor WebAssembly:** компоненти виконуються у браузері за допомогою .NET і WebAssembly.
- **Blazor Hybrid:** Razor-компоненти працюють у локальному .NET-застосунку, а інтерфейс відображається через WebView; зокрема, цей підхід використовують із .NET MAUI.

У сучасному Blazor Web App також є статичний серверний рендеринг та режим Interactive Auto. У цій роботі використовується **Interactive Server**.

Перша попередня версія Blazor вийшла 22 березня 2018 року. Реліз Blazor Server відбувся у вересні 2019 року, а Blazor WebAssembly — у травні 2020 року. Ці історичні назви допомагають орієнтуватися в старіших навчальних матеріалах.

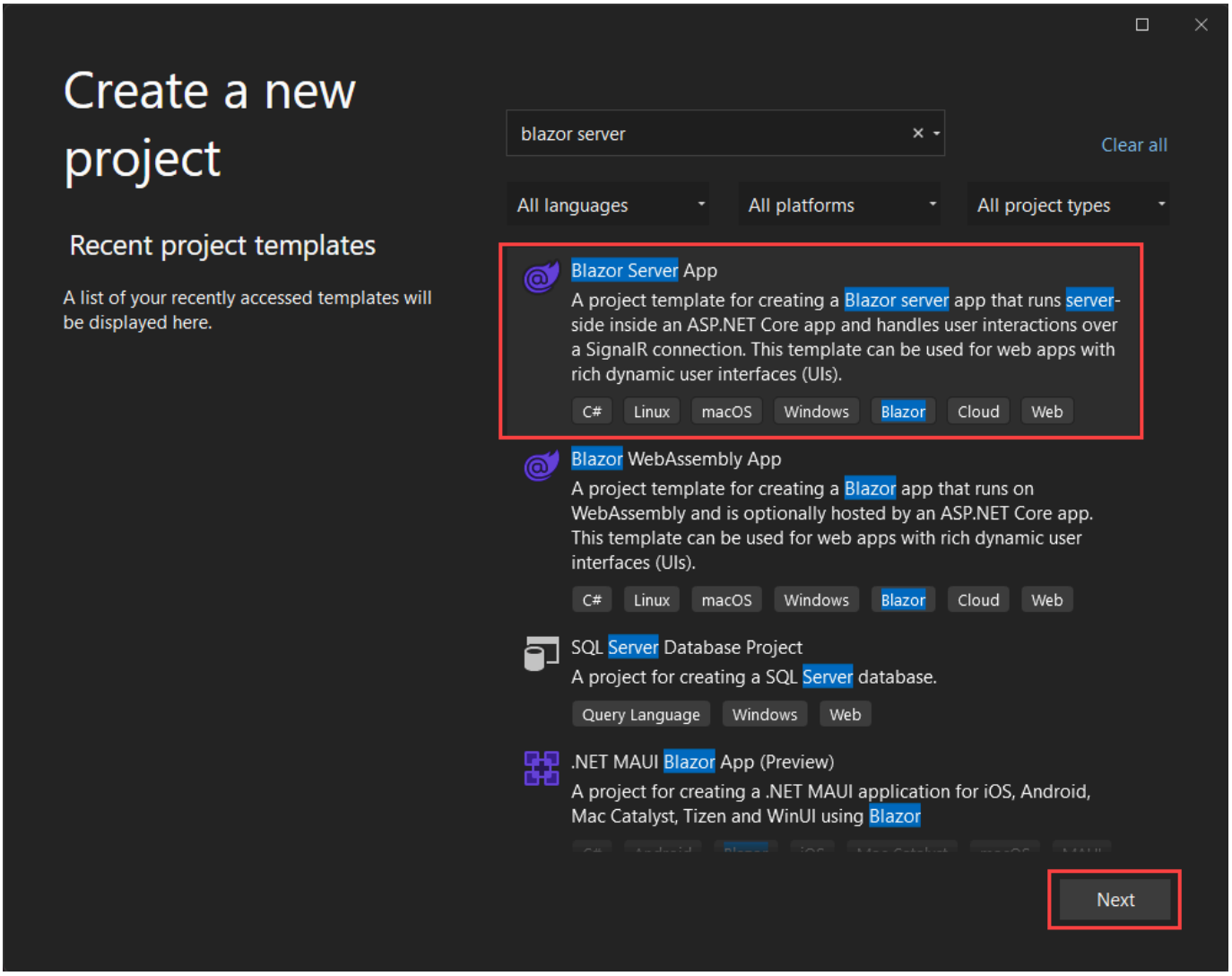
1. Створіть новий застосунок Blazor

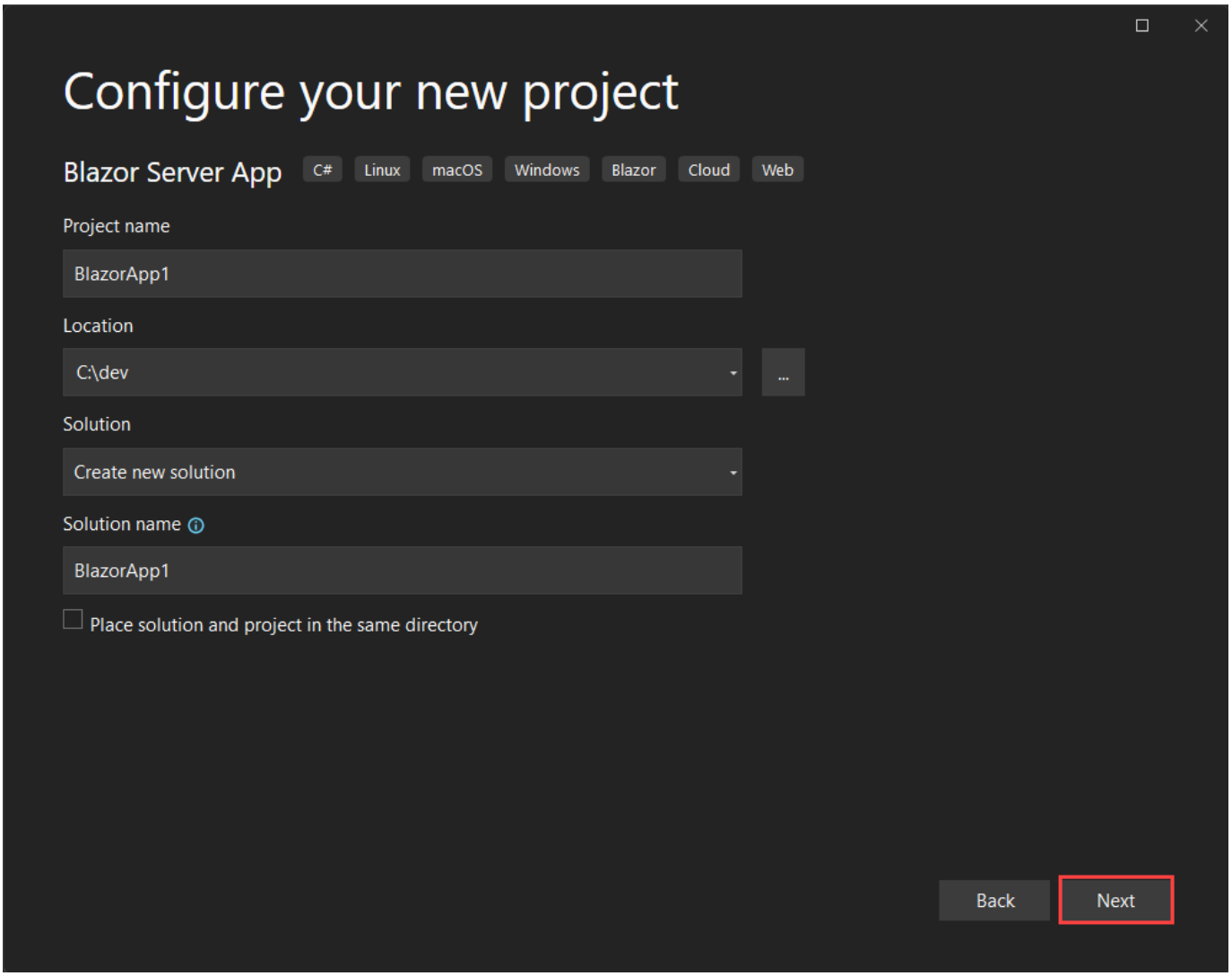
Варіант через Visual Studio

1. Виберіть **File → New → Project**.
2. Знайдіть шаблон **Blazor Web App** і натисніть **Next**.
3. Назвіть проєкт `BlazorApp`, укажіть розташування на диску й натисніть **Next**.
4. На екрані **Additional Information** виберіть .NET 10, **Authentication type: None**, **Interactive render mode: Server**, **Interactivity location: Per page/component**. Залиште HTTPS увімкненим.
5. Натисніть **Create**.

Використовуйте версію Visual Studio, яка підтримує вибрану версію .NET. Вказівка «Visual Studio 2022» на старих знімках не означає, що будь-який її випуск підходить для .NET 10.

Нижче збережено оригінальні знімки екрана. На першому показано **старий** шаблон Blazor Server App, а не сучасний Blazor Web App.





На знімку використано назву `BlazorApp1`. Для цієї інструкції задайте **BlazorApp**.

Варіант через командний рядок

Якщо використовуєте Visual Studio Code, створіть проєкт у терміналі:

```
dotnet new blazor -n BlazorApp -f net10.0 --interactivity Server
cd BlazorApp
```

Відкрийте створену папку `BlazorApp` у редакторі.

Шаблон створює базовий застосунок з необхідними компонентами та налаштуваннями.

2. Запустіть застосунок

У Visual Studio виберіть **Debug → Start Debugging** або натисніть **F5**.

У терміналі з папки проєкту виконайте:

```
dotnet watch
```

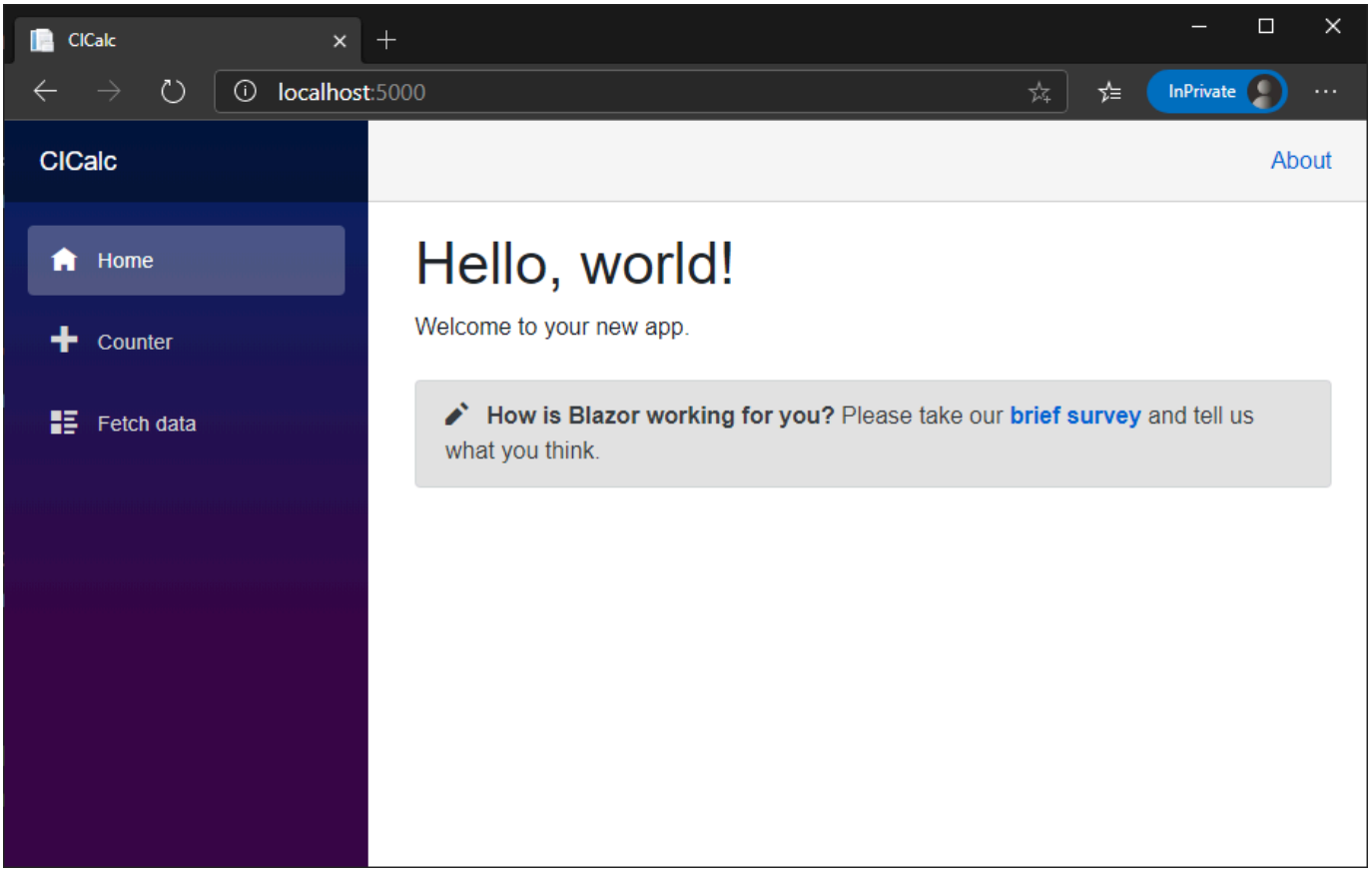
Відкрийте адресу, яку показує повідомлення запуску. Порт може відрізнятися від `5000` на зображеннях.

`dotnet watch` стежить за файлами та застосовує підтримувані зміни через Hot Reload; для інших змін може знадобитися перезапуск. Звичайний запуск із налагодженням **не гарантує** повної перебудови й перезапуску після кожного збереження. У Visual Studio застосовуйте **Hot Reload** або зупиняйте та запускайте застосунок повторно, якщо зміни не підтримуються.

Якщо під час локальної розробки є попередження про HTTPS-сертифікат, перевірте, що відкрили саме власний застосунок на `localhost`, і налаштуйте довіру до сертифіката розробки:

```
dotnet dev-certs https --trust
```

Не ігноруйте попередження про сертифікати на довільних сайтах.



3. Компоненти Razor

Дослідіть структуру проєкту та наявні сторінки.

Що таке Razor

Razor — синтаксис, який дозволяє поєднувати HTML із C#. У Blazor його використовують у файлах компонентів `.razor`.

Razor застосовується також у MVC та Razor Pages, але файли `.cshtml` і компоненти `.razor` мають різну модель роботи.

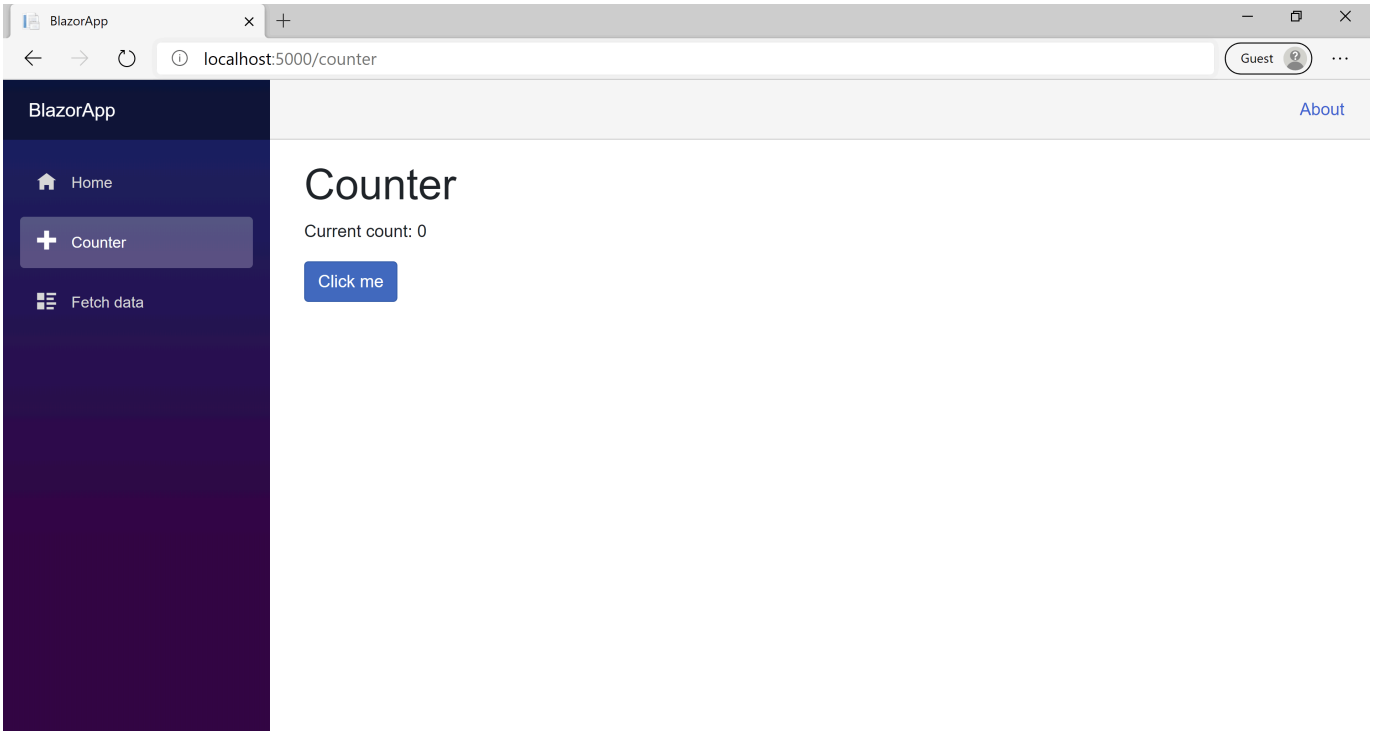
Що таке Razor-компонент

Компонент описує частину інтерфейсу: сторінку, меню, лічильник або інший елемент. Концептуально він подібний до повторно використовуваного елемента керування, хоча Blazor не використовує модель життєвого циклу ASP.NET Web Forms.

Під час збирання файл `.razor` перетворюється на .NET-клас. Компонент може містити стан, логіку рендерингу, методи життєвого циклу й обробники подій.

4. Дослідіть лічильник

У запущеному застосунку виберіть **Counter** у бічному меню.



Натисніть **Click me**. Значення збільшується без повного перезавантаження сторінки. Обробник цієї дії написаний мовою C#.

Відкрийте `Components/Pages/Counter.razor` — у старому шаблоні це `Pages/Counter.razor`. Мінімальна реалізація:

```
@page "/counter"
@rendermode InteractiveServer

<h1>Counter</h1>

<p role="status">Current count: @currentCount</p>

<button class="btn btn-primary" @onclick="IncrementCount">Click me</button>

@code {
    private int currentCount = 0;

    private void IncrementCount()
```

```
    {
        currentCount++;
    }
}
```

`@page "/counter"` пов’язує компонент з адресою `/counter`. `@rendermode InteractiveServer` дозволяє обробляти події на сервері в обраній конфігурації сучасного шаблону.

Після кожного натискання **Click me**:

1. Виникає подія натискання кнопки.
2. Blazor викликає метод `IncrementCount`.
3. Поле `currentCount` збільшується на один.
4. Компонент повторно рендериться, і на екрані з’являється нове значення.

5. Додайте компонент на головну сторінку

Відкрийте `Components/Pages/Home.razor` — у старому шаблоні це `Pages/Index.razor`.

Додайте `<Counter />` у кінець розмітки. Мінімальний вміст головної сторінки:

```
@page "/"

<h1>Hello, world!</h1>

Welcome to your new app.

<Counter />
```

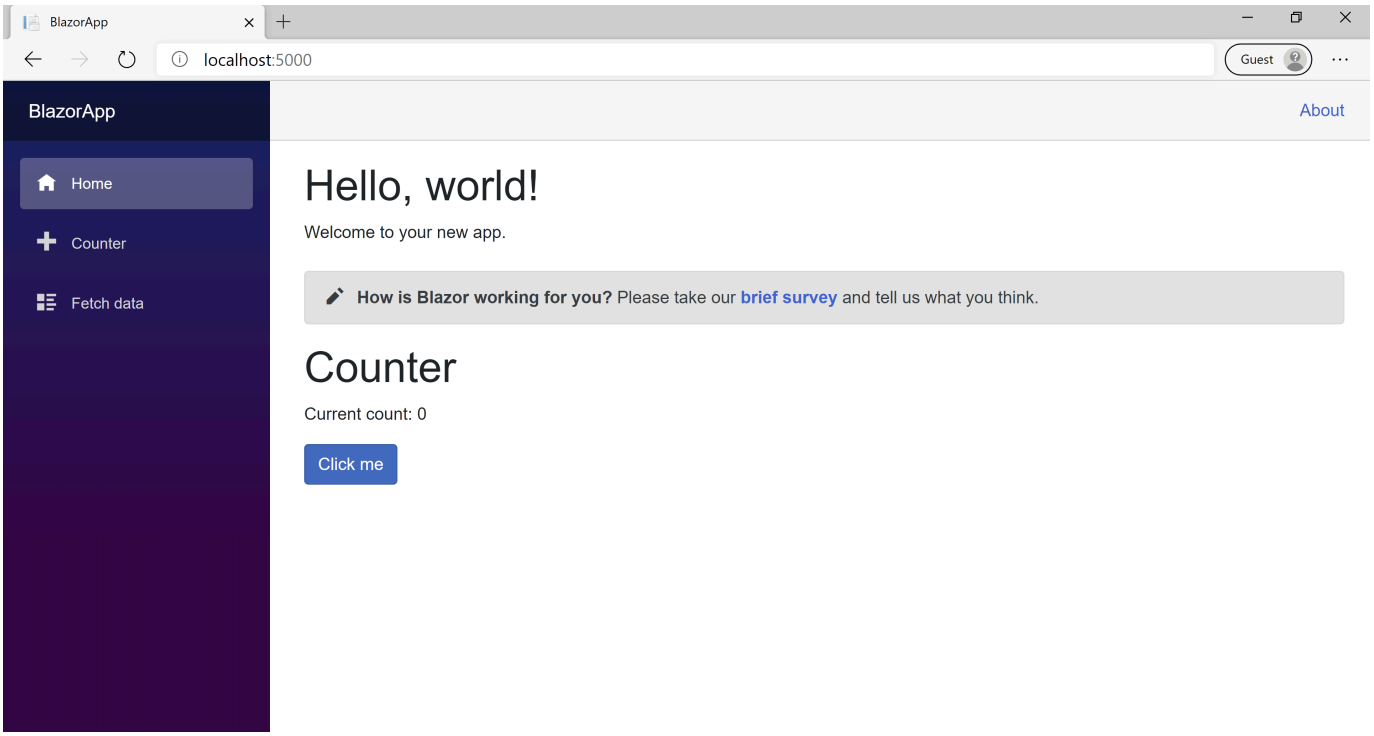
Оскільки `Counter.razor` уже містить `@rendermode InteractiveServer`, цей вкладений лічильник буде інтерактивним.

У старішій версії між привітанням і лічильником також може бути рядок:

```
<SurveyPrompt Title="How is Blazor working for you?" />
```

Залишайте його лише у старому проєкті, де є компонент `SurveyPrompt`. Сучасний шаблон його не містить, тому додавати цей рядок у новий проєкт не потрібно.

Збережіть файл і застосуйте зміни або перезапустіть застосунок. На головній сторінці має з’явитися лічильник.



6. Змініть компонент за допомогою параметра

Параметри дозволяють передавати дані від батьківського компонента до дочірнього. Зазвичай значення задають атрибутами компонента. Для передавання фрагментів інтерфейсу можна також використовувати дочірній вміст.

Додайте до лічильника параметр, який визначатиме крок збільшення:

1. Оголосіть відкриту властивість `IncrementAmount` з атрибутом `[Parameter]`.
2. У `IncrementCount` використайте її замість фіксованого збільшення на один.

Зміни, виділені зеленим у вихідному документі — `Counter.razor`:

```
     private int currentCount = 0;

+    [Parameter]
+    public int IncrementAmount { get; set; } = 1;

     private void IncrementCount()
     {
-        currentCount++;
+        currentCount += IncrementAmount;
     }
```

Повний вміст `Components/Pages/Counter.razor` після зміни:

```
@page "/counter"
@rendermode InteractiveServer

<h1>Counter</h1>
```

```
<p role="status">Current count: @currentCount</p>

<button class="btn btn-primary" @onclick="IncrementCount">Click me</button>

@code {
    private int currentCount = 0;

    [Parameter]
    public int IncrementAmount { get; set; } = 1;

    private void IncrementCount()
    {
        currentCount += IncrementAmount;
    }
}
```

Тепер у `Components/Pages/Home.razor` задайте крок `10`.

Зміна, виділена зеленим у вихідному документі — атрибут `IncrementAmount="10"`:

```
-<Counter />
+<Counter IncrementAmount="10" />
```

Повний вміст головної сторінки після зміни:

```
@page "/"

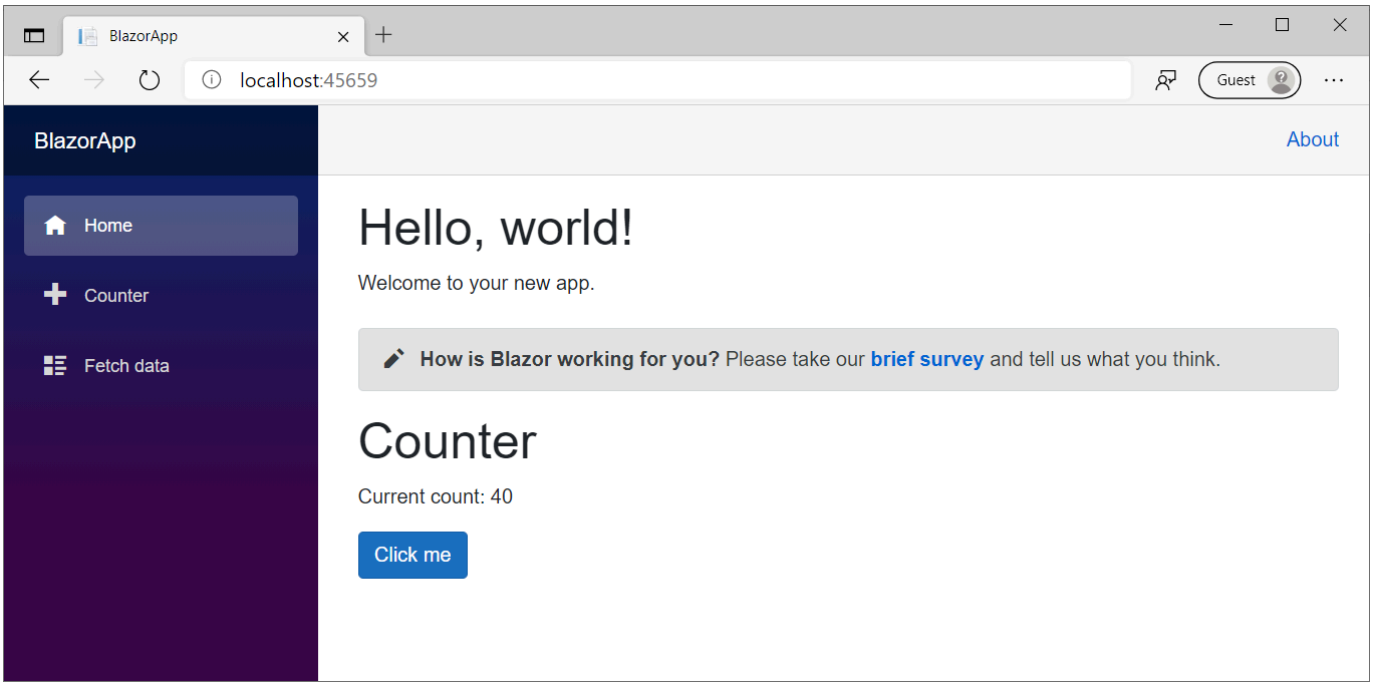
<h1>Hello, world!</h1>

Welcome to your new app.

<Counter IncrementAmount="10" />
```

На головній сторінці лічильник збільшується на `10`: `0 → 10 → 20 → 30 → 40`. На сторінці `/counter` він збільшується на `1`, оскільки там використовується значення параметра за замовчуванням.

Це різні екземпляри компонента, а не один спільний лічильник.



7. Прив'язування даних і події

Після опису інтерфейсу потрібно додати логіку. C#-код можна розміщувати безпосередньо в Razor-компоненті або в окремих файлах.

C# в окремих файлах

До Blazor-проєкту можна додавати звичайні файли `.cs`: моделі даних, сервіси та допоміжні класи.

Code-behind — конкретніший випадок: код компонента виносять у файл на зразок `Counter.razor.cs`, оголосивши відповідний `partial`-клас. Не кожен окремий `.cs`-файл є code-behind. Наприклад, `TodoItem.cs`, який створимо далі, буде моделлю даних.

Окремі файли корисні, коли логіка стає великою або використовується в кількох місцях. Для простої логіки створювати додатковий файл не обов'язково.

C# у Razor-компоненті

У невеликому компоненті зручно поєднати розмітку й код в одному `.razor`-файлі.

Razor використовує директиви, вирази та блоки керування. Це не одне й те саме:

- `@currentCount` і `@(currentCount + 1)` — вирази, значення яких виводяться в розмітку;
- `@if` і `@foreach` — умовне формування та повторення розмітки;
- `@code { ... }` — поля, властивості й методи класу компонента;
- `@page` — директива маршруту;
- `@rendermode` — директива режиму рендерингу.

Блок `@code` не призначений для довільних операторів на рівні класу: виконувані дії розміщують усередині методів. Для оголошення членів класу також існує `@functions`, але в Blazor-компонентах

зазвичай використовують `@code`.

Директива `@page`

`@page` надає компоненту маршрут. Наприклад, `@page "/todo"` дозволяє відкрити його за адресою `/todo`. Вкладеному компоненту, який не має окремої адреси, ця директива не потрібна.

Як працює `@bind`

Прив'язування даних синхронізує значення елемента інтерфейсу з полем або властивістю C#.

Під час рендерингу значення передається **з C# до інтерфейсу**. Коли користувач змінює поле і виникає відповідна подія, нове значення надходить **з інтерфейсу до C#**. Для звичайного текстового `<input>` директива `@bind` за замовчуванням використовує `onchange`, а не кожне натискання клавіші.

Після звичайного обробника події Blazor сам ініціює оновлення інтерфейсу. Приклад прив'язування наведено в наступному завданні. Документація щодо прив'язування даних.

8. Створіть сторінку Todo

Створіть основний компонент списку завдань.

У Visual Studio клацніть правою кнопкою миші папку `Components/Pages` у **Solution Explorer**, виберіть **Add → Razor Component** і задайте ім'я `Todo.razor`. У Visual Studio Code створіть цей файл у тій самій папці.

Важливо: назва Razor-компонента має починатися з великої літери: `Todo.razor`, а не `todo.razor`.

Додайте маршрут і ввімкніть серверну інтерактивність:

```
@page "/todo"
@rendermode InteractiveServer

<h3>Todo</h3>

@code {

}
```

Збережіть `Components/Pages/Todo.razor`. Не створюйте другий компонент із тим самим маршрутом `/todo`.

9. Додайте Todo до меню

`NavMenu` використовується у спільному макеті застосунку. Макет дозволяє не дублювати однакову структуру на кожній сторінці.

`NavLink` — компонент посилання, який може позначати активний пункт меню залежно від поточної адреси.

Відкрийте `Components/Layout/NavMenu.razor` — у старому шаблоні це `Shared/NavMenu.razor`. Усередині наявного `<nav>...</nav>` додайте:

```
<div class="nav-item px-3">
    <NavLink class="nav-link" href="todo">
        Todo
    </NavLink>
</div>
```

Не замінюйте весь `NavMenu.razor` цим фрагментом.

Дляч старішої версії .Net:

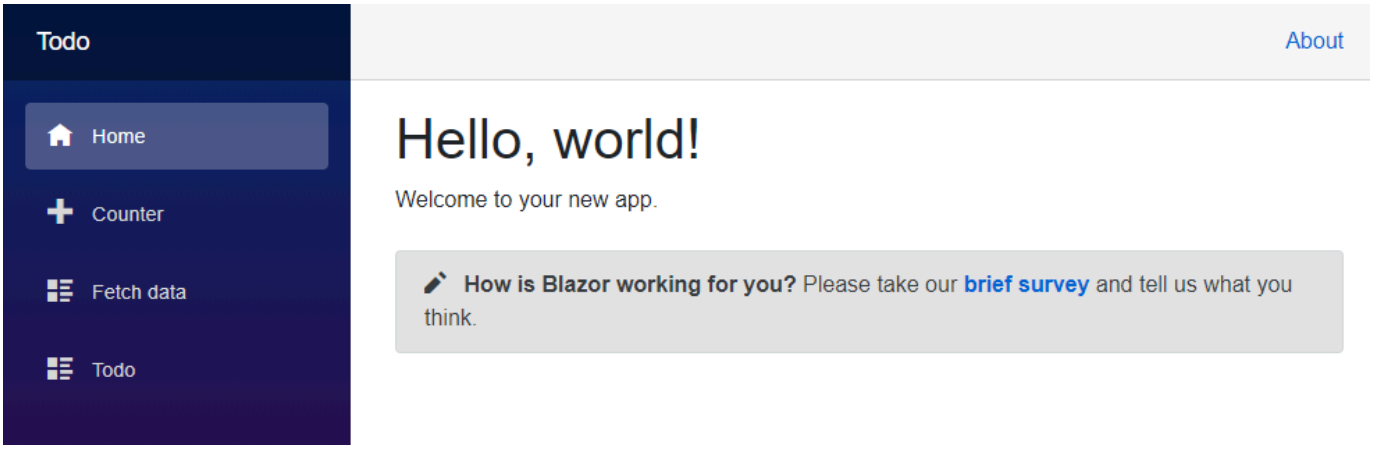
```
<div class="@NavMenuCssClass" @onclick="ToggleNavMenu">
    <nav class="flex-column">

        @* Тут залишаються інші наявні пункти меню. *@

        <div class="nav-item px-3">
            <NavLink class="nav-link" href="todo">
                <span class="oi oi-list-rich" aria-hidden="true"></span> Todo
            </NavLink>
        </div>
    </nav>
</div>
```

Цей варіант стосується лише старого шаблону, у якому вже визначено `NavMenuCssClass`, `ToggleNavMenu` та стилі Open Iconic (`oi`). У сучасному шаблоні залиште його власну оболонку меню та додайте тільки новий пункт. Піктограма необов'язкова.

Збережіть файл і застосуйте зміни. У меню має з'явитися **Todo**.



10. Створіть клас TodoItem

У корені проєкту `BlazorApp`, поряд із файлом `.csproj`, створіть `TodoItem.cs`.

Цей клас представлятиме одне завдання:

```
public class TodoItem
{
    public string? Title { get; set; }
    public bool IsDone { get; set; } = false;
}
```

`Title` — назва завдання. Позначення `string?` дозволяє значення `null`. `IsDone` — логічна ознака виконання: `true` або `false`.

Для точного відтворення цього прикладу замініть увесь згенерований вміст `TodoItem.cs` наведеним кодом без `namespace`. Якщо залишаєте простір імен, забезпечте його доступність через `@using` у компоненті або `_Imports.razor`.

11. Відобразіть список завдань

У `Components/Pages/Todo.razor`:

1. Додайте в `@code` поле `todos` для зберігання об'єктів `TodoItem`.
2. Додайте невпорядкований HTML-список `<ul>`.
3. За допомогою `@foreach` сформуйте `<li>` для кожного завдання.

```
@page "/todo"
@rendermode InteractiveServer

<h3>Todo</h3>

<ul>
    @foreach (var todo in todos)
    {
        <li>@todo.Title</li>
    }
</ul>

@code {
    private List<TodoItem> todos = new();
}
```

`new()` створює порожній список. Тому спочатку на сторінці ще немає жодного пункту.

12. Додайте поле введення та кнопку

Під списком `<ul>...</ul>` розмістіть текстове поле й кнопку:

```
@page "/todo"
@rendermode InteractiveServer

<h3>Todo</h3>
```

```
<ul>
    @foreach (var todo in todos)
    {
        <li>@todo.Title</li>
    }
</ul>

<input placeholder="Something to do" />
<button>Add todo</button>

@code {
    private List<TodoItem> todos = new();
}
```

На цьому кроці натискання **Add todo** нічого не додає: кнопка ще не має обробника події.

13. Підключіть обробник події

Додайте метод `AddTodo` до наявного блоку `@code` і зв'яжіть його з кнопкою через `@onclick`.

Нижче показано фрагмент для заміни поля, кнопки та блоку `@code`. Маршрут, режим, заголовок і список вище залишаються:

```
<input placeholder="Something to do" />
<button @onclick="AddTodo">Add todo</button>

@code {
    private List<TodoItem> todos = new();

    private void AddTodo()
    {
        // TODO: додати завдання до списку.
    }
}
```

Тепер натискання викликає `AddTodo`, але метод поки що порожній. Не додавайте друге поле `todos`, якщо воно вже є.

14. Зв'яжіть введений текст із C#

Додайте до наявного блоку `@code` поле `newTodo`:

```
private List<TodoItem> todos = new();
private string? newTodo;

// Далі залишається метод AddTodo.
```

Перший рядок уже є у файлі — повторно його не вставляйте. Новим тут є `private string? newTodo;`.

Змініть текстове поле:

```
<input placeholder="Something to do" @bind="newTodo" />
```

Тепер `@bind` пов’язує значення поля введення з `newTodo`. Коли користувач завершує редагування поля і виникає `onchange`, значення оновлюється в C#.

15. Реалізуйте додавання завдання

Оновіть `AddTodo`: перевірте введений текст, створіть `TodoItem`, додайте його до списку й очистіть поле введення.

Повний вміст `Components/Pages/Todo.razor` на цьому кроці:

```
@page "/todo"
@rendermode InteractiveServer

<h3>Todo</h3>

<ul>
    @foreach (var todo in todos)
    {
        <li>@todo.Title</li>
    }
</ul>

<input placeholder="Something to do" @bind="newTodo" />
<button @onclick="AddTodo">Add todo</button>

@code {
    private List<TodoItem> todos = new();
    private string? newTodo;

    private void AddTodo()
    {
        if (!string.IsNullOrWhiteSpace(newTodo))
        {
            todos.Add(new TodoItem { Title = newTodo });
            newTodo = string.Empty;
        }
    }
}
```

`string.IsNullOrWhiteSpace` перевіряє, чи значення є `null`, порожнім рядком або складається лише з пробільних символів. Знак `!` заперечує результат, тому завдання додається лише за наявності змістовного тексту.

`newTodo = string.Empty` очищує поле C#, а прив’язування оновлює текстове поле на екрані.

Збережіть файл і застосуйте зміни через Hot Reload або перезапуск. Додайте кілька завдань.

16. Додайте редагування та позначку виконання

Замість простого виведення назви додайте:

- прапорець, пов'язаний із `todo.IsDone`;
- текстове поле, пов'язане з `todo.Title`.

Замініть лише наявний блок `<ul>...</ul>`:

```
<ul>
    @foreach (var todo in todos)
    {
        <li>
            <input type="checkbox" @bind="todo.IsDone" />
            <input @bind="todo.Title" />
        </li>
    }
</ul>
```

Назви завдань тепер можна редагувати, а прапорці змінюють значення `IsDone`. Решту коду залиште без змін.

17. Покажіть кількість незавершених завдань

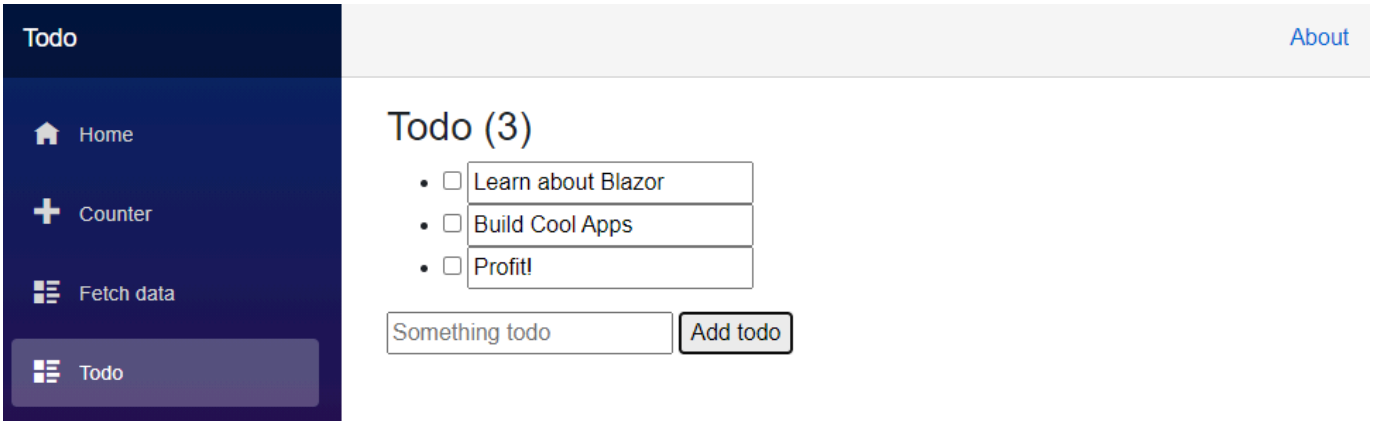
Замініть заголовок:

```
<h3>Todo (@todos.Count(todo => !todo.IsDone))</h3>
```

`Count(...)` підраховує елементи, які відповідають умові. Вираз `todo => !todo.IsDone` означає: врахувати завдання, для якого `IsDone` дорівнює `false`.

Метод `Count` із такою умовою є методом LINQ. У стандартному шаблоні вже є `@using System.Linq`; якщо цей імпорт було видалено, поверніть його в `Components/_Imports.razor`.

Збережіть файл, застосуйте зміни й перевірте компонент: додавайте завдання, редагуйте назви та позначайте завдання як виконані. Кількість у заголовку має зменшуватися після встановлення прапорця.

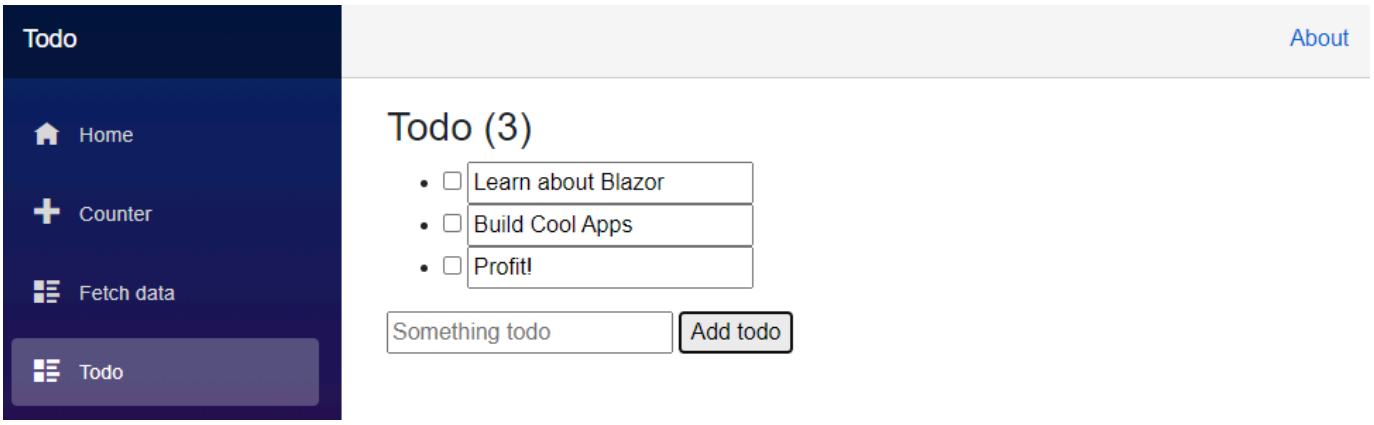


Важливо: список зберігається лише у стані компонента, а не в базі даних. Створення нового екземпляра компонента або повний перезапуск застосунку може очистити дані. Постійне збереження не входить до цього прикладу.

Після завершення основного прикладу виконайте самостійну частину: додайте сторінку з обраним інструментом, посилання на неї в меню та власне CSS-оформлення.

Завдання

1. Розробіть вебзастосунок Blazor за наведеними вище кроками. Результатом має бути застосунок зі списком завдань.



2. У створеному застосунку самостійно додайте нову сторінку, доступну через бічне меню. Використовуючи HTML і C#, реалізуйте один із запропонованих варіантів:

    - **Калькулятор:** додавання, віднімання, ділення та множення двох чисел. Передбачте перевірку ділення на нуль.
    - **Конвертер фізичних величин:** температура за шкалами Цельсія і Фаренгейта, об'єм, сила, маса, енергія тощо.
    - **Опитувальник — Mini Quiz:** запитання з кількома варіантами відповіді, перемикачі вибору та кнопка «Перевірити відповідь».
    - **Генератор паролів:** кнопка «Згенерувати пароль», вибір довжини та набору символів — лише цифри або літери, цифри й спеціальні символи. Якщо пароль призначений для реального використання, застосуйте криптографічно стійкий генератор випадкових значень, а не звичайний `Random`.
    - **Пошук у тексті:** поле введення, з якого після натискання кнопки виділяються адреси електронної пошти, вебадреси, номери телефонів або інший текст за заданим шаблоном. Знайдені значення виводяться окремим списком.

3. Змініть вигляд застосунку за допомогою CSS.

Використані ресурси

- Вихідний навчальний модуль Microsoft Learn — Build your first Blazor web app.
- Уточнення для сучасного шаблону — Build a Blazor todo list app.
- Режими рендерингу Blazor.

- Прив’язування даних у Blazor.

Примітки до виправлень

- Виправлено орфографію, пунктуацію, термінологію та назву поля `currentCount` з урахуванням регістру.
- Уточнено різницю між старим Blazor Server App і сучасним Blazor Web App; оригінальні знімки екрана збережено.
- Для сучасного варіанта вказано відповідні шляхи й додано серверну інтерактивність. Старі `SurveyPrompt`, `ToggleNavMenu` та піктограми `oi` позначено як залежні від шаблону.
- Виправлено пояснення Hot Reload, code-behind, `@code`, `@(...)` та напрямків передавання даних у `@bind`.
- Прибрано безумовну пораду ігнорувати попередження HTTPS. Уточнено, що список не має постійного сховища.
- Зелені фрагменти оригіналу передано двома блоками `diff`; звичайне синтаксичне забарвлення слів у коді не трактувалося як позначення змін.